

Binary Search Trees: Introduction

Daniel Kane

Department of Computer Science and Engineering
University of California, San Diego

Data Structures
Data Structures and Algorithms

Learning Objectives

- Provide examples of the sorts of problems we hope to solve with Binary Search Trees.
- Show why data structures that we have already covered are insufficient.

Outline

1 Local Search

2 Attempts

Dictionary Search

Find all words that start with some given string.



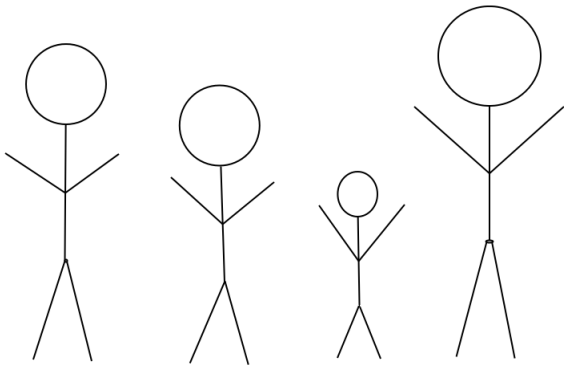
Date Ranges

Find all emails received in a given period.

Inbox				
FROM	KNOW	TO	SUBJECT	SENT TIME ▼
"lawiki.i2p admin" <J5uF>		Bote User <uhOd>	hi	Unknown
anonymous		Bote User <uhOd>	Sanders 2016	Aug 30, 2015 3:27 PM
anonymous		Bote User <uhOd>	I2PCon 2016	Aug 30, 2015 3:25 PM
Anon Developer <gvbM>		Bote User <uhOd>	Re: Bote changess	Aug 30, 2015 2:54 PM
I2P User <uUUX>		Bote User <uhOd>	Hello World!	Aug 30, 2015 2:51 PM

Closest Height

Find the person in your class whose height is closest to yours.



Local Search

Definition

A **Local Search Datastructure** stores a number of elements each with a **key** coming from an ordered set. It supports operations:

- **RangeSearch(x, y)**: Returns all elements with keys between x and y .
- **NearestNeighbors(z)**: Returns the element with keys on either side of z .

Example

1	4	6	7	10	13	15
---	---	---	---	----	----	----

Example

1	4	6	7	10	13	15
---	---	---	---	----	----	----

RangeSearch(5, 12)

1	4	6	7	10	13	15
---	---	---	---	----	----	----

Example

1	4	6	7	10	13	15
---	---	---	---	----	----	----

RangeSearch(5, 12)

1	4	6	7	10	13	15
---	---	---	---	----	----	----

NearestNeighbors(3)

1	4	6	7	10	13	15
---	---	---	---	----	----	----

Dynamic Data Structure

We would also like to be able to modify the data structure as we go.

- `Insert( $x$ )`: Adds a element with key x .
- `Delete( $x$ )`: Removes the element with key x .

Example

1	4	6	7	10	13	15
---	---	---	---	----	----	----

Example

1	4	6	7	10	13	15
---	---	---	---	----	----	----

Insert(3)

1	3	4	6	7	10	13	15
---	---	---	---	---	----	----	----

Example

1	4	6	7	10	13	15
---	---	---	---	----	----	----

Insert(3)

1	3	4	6	7	10	13	15
---	---	---	---	---	----	----	----

Delete(10)

1	3	4	6	7	13	15
---	---	---	---	---	----	----

Problem

If an empty data structure is given these commands what does it output at the end?

- Insert(3)
- Insert(8)
- Insert(5)
- Insert(10)
- Delete(8)
- Insert(12)
- NearestNeighbors(7)

Answer

3	5	8	10	12
---	---	--------------	----	----

Outline

① Local Search

② Attempts

Array

■ RangeSearch:

$O(n)$ ✗

7	10	4	13	1	6	15
---	----	---	----	---	---	----

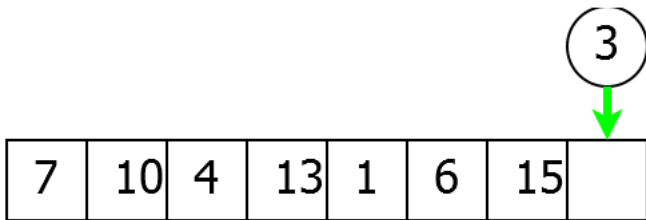
Array

- RangeSearch: $O(n)$ ✗
- NearestNeighbors: $O(n)$ ✗

7	10	4	13	1	6	15
---	----	---	----	---	---	----

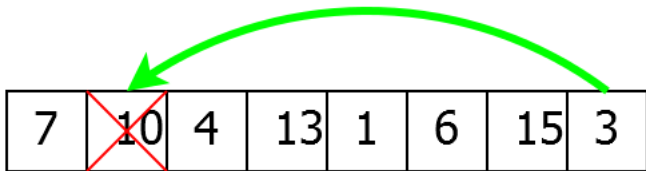
Array

- RangeSearch: $O(n)$ ✗
- NearestNeighbors: $O(n)$ ✗
- Insert: $O(1)$ ✓



Array

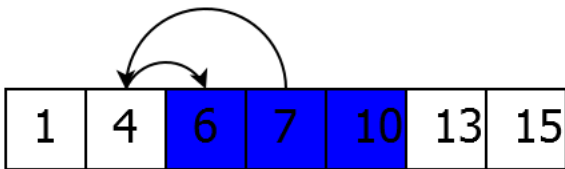
- RangeSearch: $O(n)$ ✗
- NearestNeighbors: $O(n)$ ✗
- Insert: $O(1)$ ✓
- Delete: $O(1)$ ✓



Sorted Array

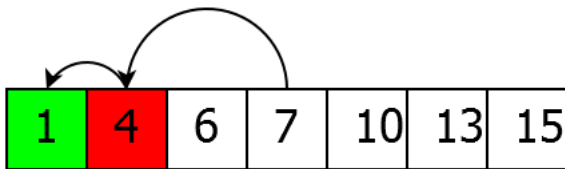
■ RangeSearch:

$O(\log(n))$ ✓



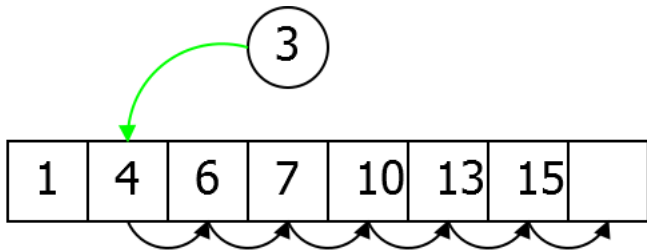
Sorted Array

- RangeSearch: $O(\log(n))$ ✓
- NearestNeighbors: $O(\log(n))$ ✓



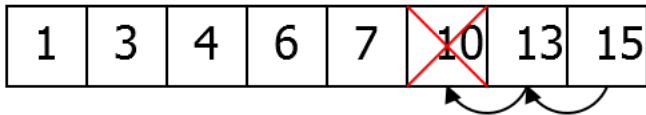
Sorted Array

- RangeSearch: $O(\log(n))$ ✓
- NearestNeighbors: $O(\log(n))$ ✓
- Insert: $O(n)$ ✗



Sorted Array

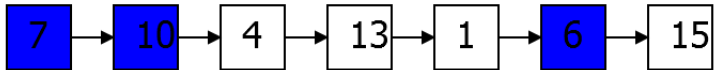
- RangeSearch: $O(\log(n))$ ✓
- NearestNeighbors: $O(\log(n))$ ✓
- Insert: $O(n)$ ✗
- Delete: $O(n)$ ✗



Linked List

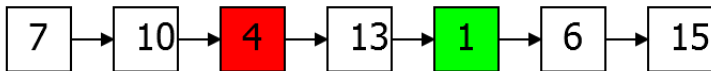
■ RangeSearch:

$O(n)$ ✗



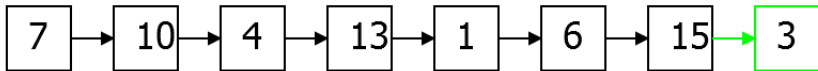
Linked List

- RangeSearch: $O(n)$ ✗
- NearestNeighbors: $O(n)$ ✗



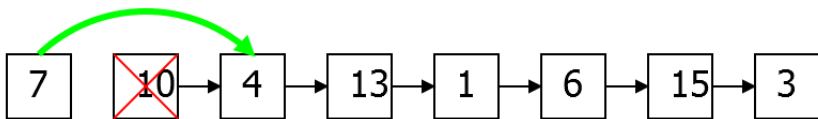
Linked List

- RangeSearch: $O(n)$ ✗
- NearestNeighbors: $O(n)$ ✗
- Insert: $O(1)$ ✓



Linked List

- RangeSearch: $O(n)$ ✗
- NearestNeighbors: $O(n)$ ✗
- Insert: $O(1)$ ✓
- Delete: $O(1)$ ✓



Need Something New

Problem

Previous data structures won't work. We need something new.

Binary Search Trees: Search Trees

Daniel Kane

Department of Computer Science and Engineering
University of California, San Diego

Data Structures
Data Structures and Algorithms

Learning Objectives

- Describe how a Binary Search Tree data structure is constructed.
- Determine whether a tree is properly sorted.

Last Time

- Want data structure for local search.

Last Time

- Want data structure for local search.
- None of the existing data structures work.

Last Time

- Want data structure for local search.
- None of the existing data structures work.
- Sorted arrays can search but not update.

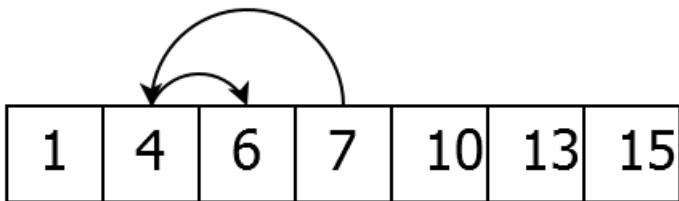
Outline

① Array Search

② The Search Tree Structure

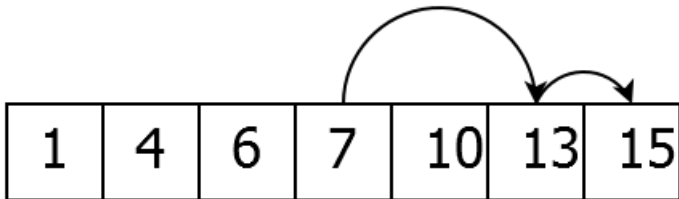
Binary Search

Search an array:



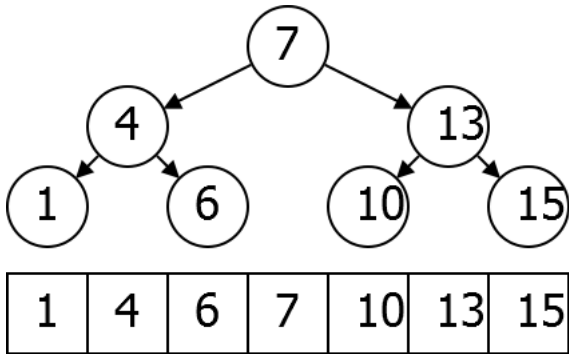
Binary Search

Search an array:



Search Tree

Consider questions asked:



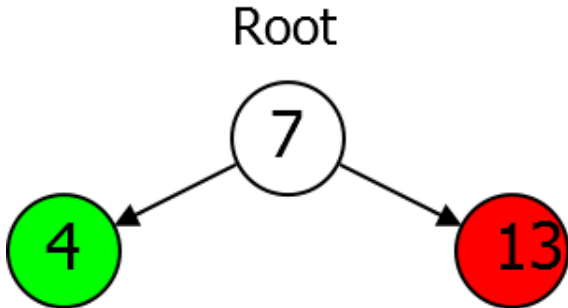
The search tree is much easier to insert into.

Outline

- 1 Array Search
- 2 The Search Tree Structure

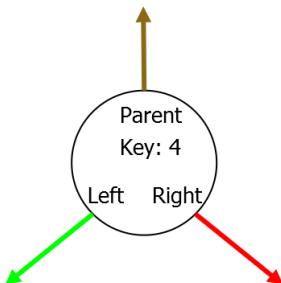
Parts of a Tree

- Root node.
- Left subtree smaller keys.
- Right subtree bigger keys.



Tree Node Data Type

- Key
- Parent
- Left Child
- Right Child

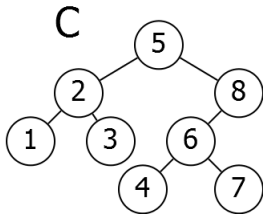
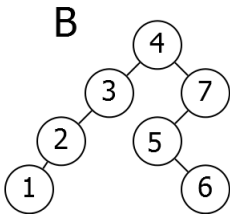
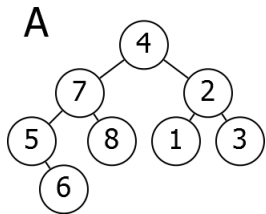


Search Tree Property

X 's key is larger than the key of any descendent of its left child, and smaller than the key of any descendant of its right child.

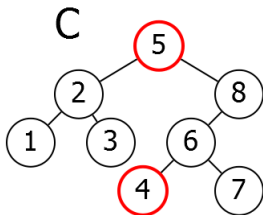
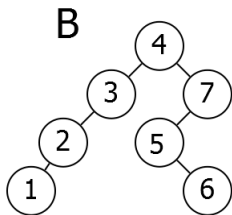
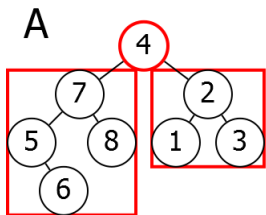
Problem

Which of the following Trees satisfies the Search Tree Property?



Problem

Which of the following Trees satisfies the Search Tree Property?



Next Time

How to do basic operations on Binary Search Trees.

Binary Search Trees: Basic Operations

Daniel Kane

Department of Computer Science and Engineering
University of California, San Diego

Data Structures
Data Structures and Algorithms

Learning Objectives

- Implement basic operations on Binary Search Trees.
- Understand some of the difficulties with making updates.

Outline

- 1 Find
- 2 Next Element
- 3 Search
- 4 Insert
- 5 Delete

Find

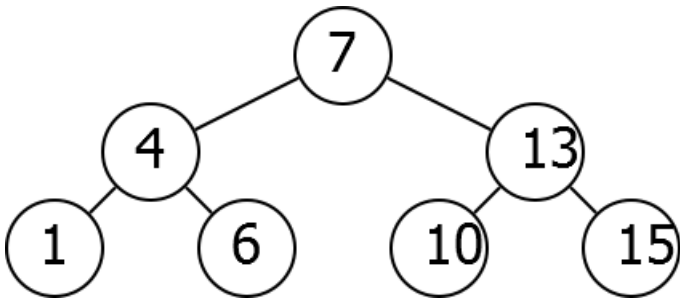
Find

Input: Key k , Root R

Output: The node in the tree of R with key k

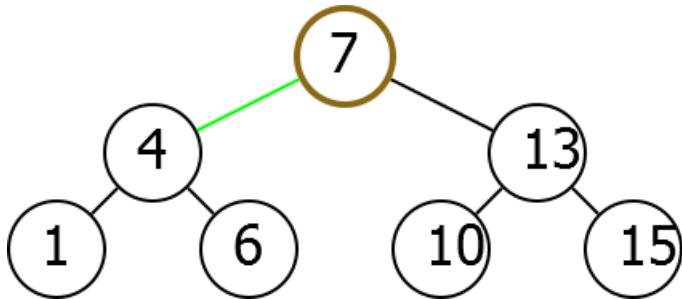
Idea

Find(6)



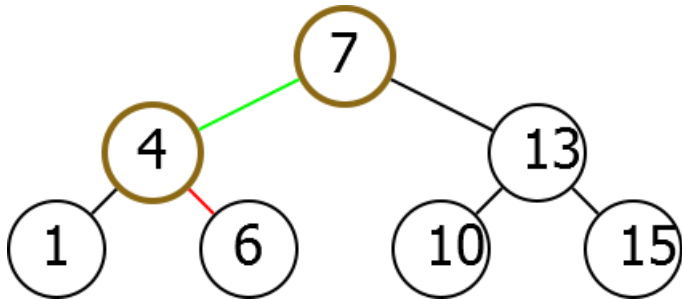
Idea

Find(6)



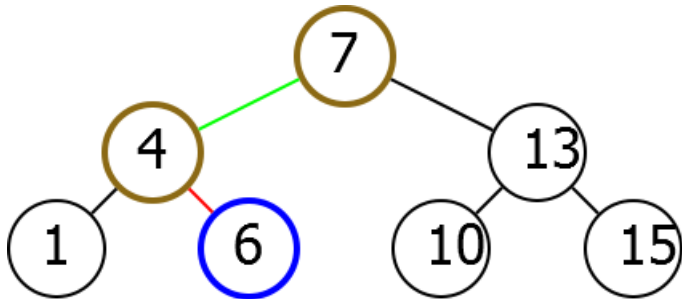
Idea

Find(6)



Idea

Find(6)



Algorithm

Find(k, R)

if $R.\text{Key} = k$:

 return R

else if $R.\text{Key} > k$:

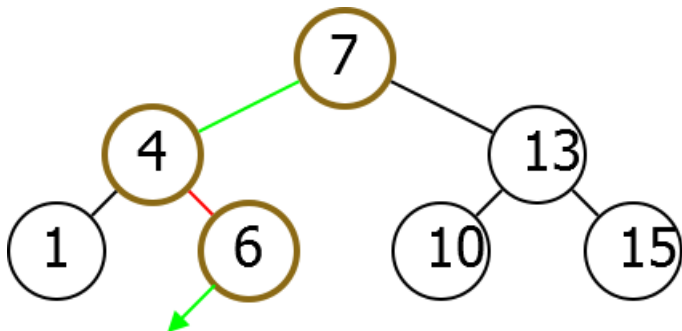
 return Find($k, R.\text{Left}$)

else if $R.\text{Key} < k$:

 return Find($k, R.\text{Right}$)

Missing Key

Run Find(5).



Key not in tree. Did find point where it should be.

Missing Key

If you stop before reaching a null pointer, you find the place in the tree where k would fit.

Modification

Find (modified)

```
else if  $R.\text{Key} > k$  :  
    if  $R.\text{Left} \neq \text{null}$ :  
        return Find( $k, R.\text{Left}$ )  
    return  $R$ 
```

Outline

- 1 Find
- 2 Next Element
- 3 Search
- 4 Insert
- 5 Delete

Adjacent Elements

Given a node N in a Binary Search Tree, would like to find adjacent elements.

Next

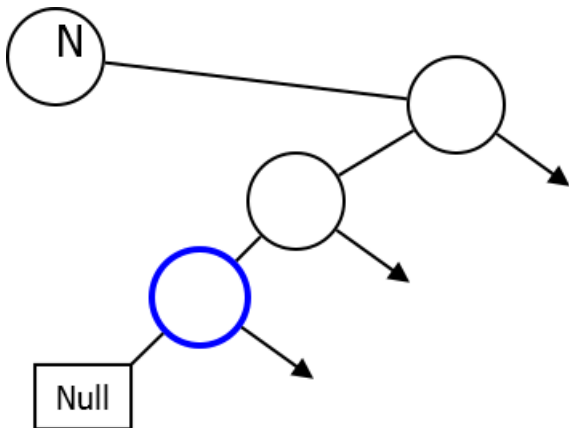
Next

Input: Node N

Output: The node in the tree with the next largest key.

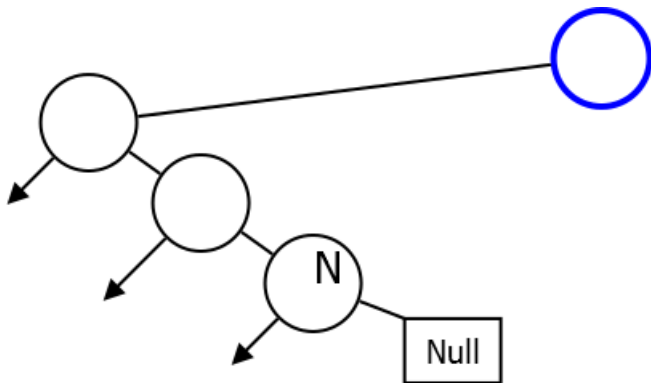
Case I

If you have right child.



Case II

No right child.



Next

Next(*N*)

```
if N.Right  $\neq$  null:  
    return LeftDescendant(N.Right)  
else:  
    return RightAncestor(N)
```


Left Descendant

LeftDescendant(*N*)

```
if N.Left = null
    return N
else:
    return LeftDescendant(N.Left)
```

Right Ancestor

RightAncestor(N)

```
if  $N.Key < N.Parent.Key$   
    return  $N.Parent$   
else:  
    return RightAncestor( $N.Parent$ )
```

Outline

- 1 Find
- 2 Next Element
- 3 Search
- 4 Insert
- 5 Delete

Range Search

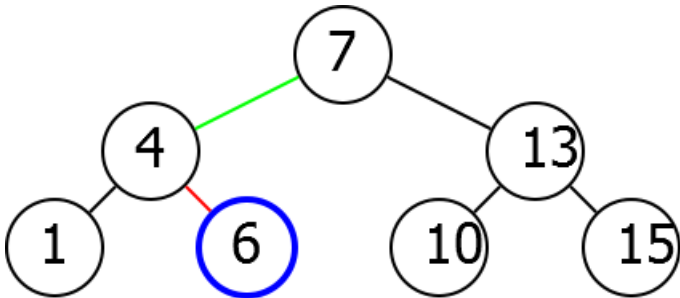
Range Search

Input: Numbers x, y , root R

Output: A list of nodes with key between x and y

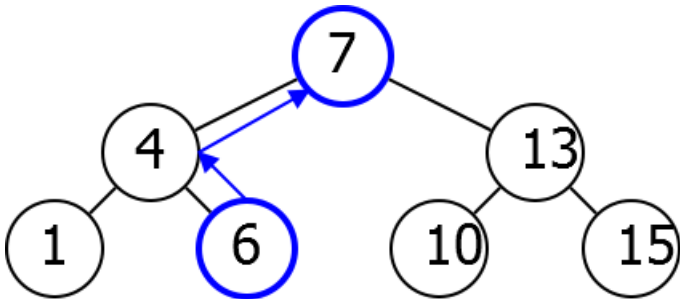
Idea

RangeSearch(5, 12).



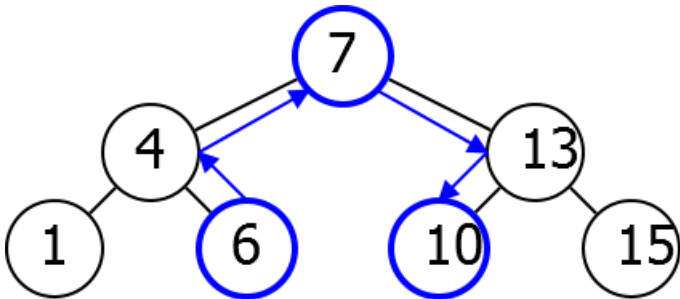
Idea

RangeSearch(5, 12).



Idea

RangeSearch(5, 12).



Implementation

RangeSearch(x, y, R)

$L \leftarrow \emptyset$

$N \leftarrow \text{Find}(x, R)$

while $N.\text{Key} \leq y$

 if $N.\text{Key} \geq x$:

$L \leftarrow L.\text{Append}(N)$

$N \leftarrow \text{Next}(N)$

return L

Outline

- 1 Find
- 2 Next Element
- 3 Search
- 4 Insert
- 5 Delete

Insert

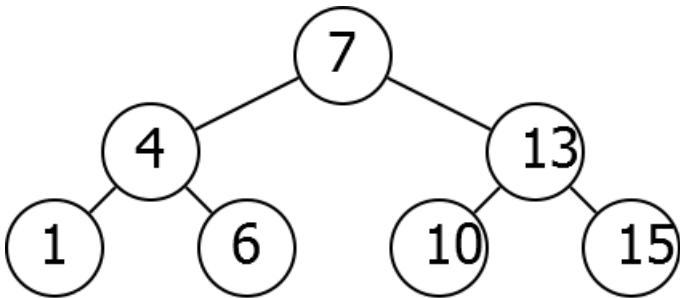
Insert

Input: Key k and root R

Output: Adds node with key k to the tree

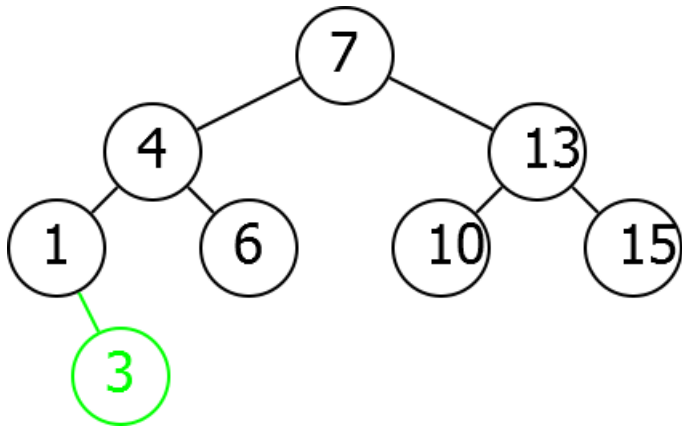
Insert Idea

Insert(3)



Insert Idea

Insert(3)



Implementation

$\text{Insert}(k, R)$

$P \leftarrow \text{Find}(k, R)$

Add new node with key k as child of
 P

Outline

- 1 Find
- 2 Next Element
- 3 Search
- 4 Insert
- 5 Delete

Delete

Delete

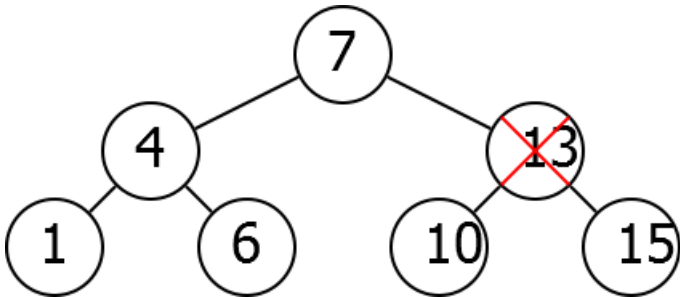
Input: Node N

Output: Removes node N from the tree

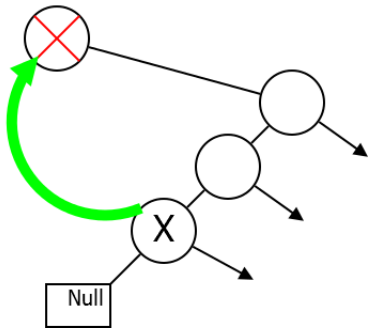
Difficulty

Cannot simply remove.

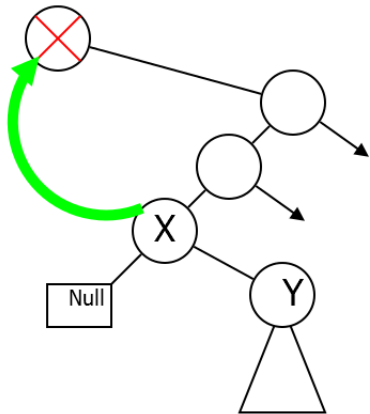
Delete(13)



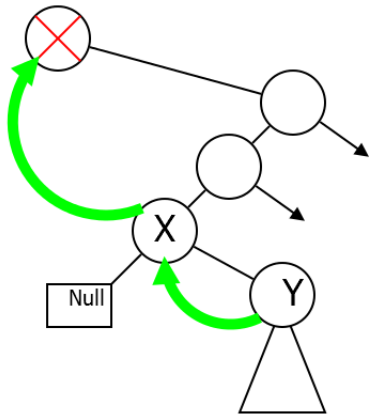
Idea



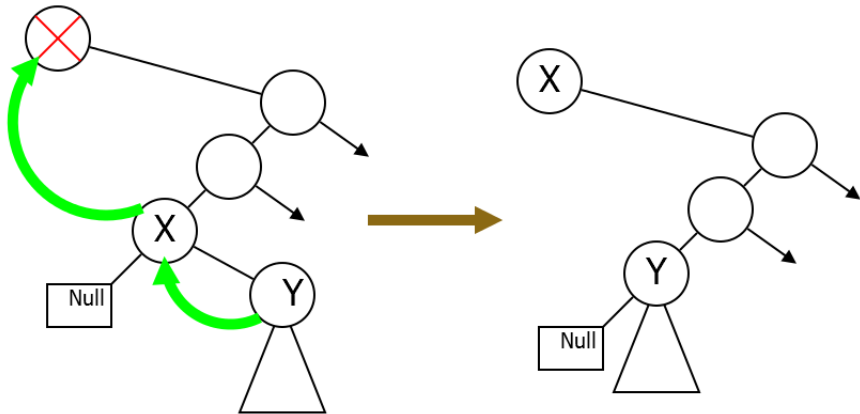
Idea



Idea



Idea



Implementation

Delete(N)

```
if  $N.Right = \text{null}$ :
```

```
    Remove  $N$ , promote  $N.Left$ 
```

```
else:
```

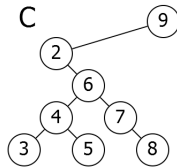
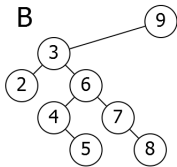
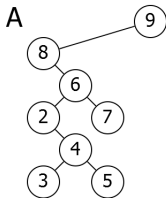
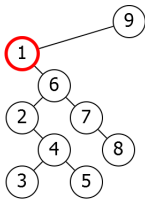
```
     $X \leftarrow \text{Next}(N)$ 
```

```
     $X.Left = \text{null}$ 
```

```
    Replace  $N$  by  $X$ , promote  $X.Right$ 
```

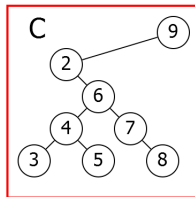
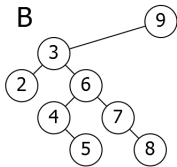
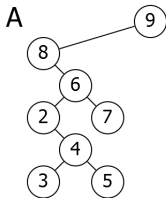
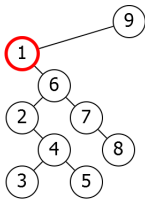
Problem

Which of the following trees is obtained when the selected node is deleted?



Problem

Which of the following trees is obtained when the selected node is deleted?



Next Time

Runtime and balance.